# Agentic Document Workflows

---

RAG, Workflow, Event-Driven, Agent, LlamaIndex, Human-in-the-Loop

**Hanwha Life Vietnam**

Phiên bản v1.0.0

Ngày 12 tháng 10 năm 2026

# Mục lục


**1 Tổng quan** **2**
- 1.1 Triết lý trình bày . . . 2
- 1.2 Cách đọc tài liệu . . . 2
- 1.3 Nội dung các chương . . . 2
- 1.4 Nguồn và quy ước . . . 3

**2 Khái niệm Agentic Document Workflows** **4**
- 2.1 RAG là gì . . . 4
  - 2.1.1 Embedding và vector space . . . 4
- 2.2 Giới hạn của RAG . . . 5
- 2.3 Agent và workflow . . . 5
  - 2.3.1 Agent . . . 5
  - 2.3.2 Workflow . . . 5
- 2.4 Agentic Document Workflows . . . 5

**3 Xây dựng workflow** **7**
- 3.1 Workflow một bước . . . 7
- 3.2 Workflow nhiều bước . . . 8
- 3.3 Vòng lặp . . . 9
- 3.4 Rẽ nhánh . . . 9
- 3.5 Chạy song song . . . 10
  - 3.5.1 Gom nhiều sự kiện cùng loại . . . 10
  - 3.5.2 Gom sự kiện khác loại . . . 11
- 3.6 Truyền sự kiện theo luồng (streaming) . . . 11

**4 Thêm RAG** **13**
- 4.1 Parse CV bằng LlamaParse . . . 13
- 4.2 Dựng vector store và query engine . . . 14
  - 4.2.1 Lưu và nạp lại vector store . . . 14
- 4.3 Biến RAG thành công cụ cho agent . . . 15
- 4.4 Đóng gói thành workflow . . . 15

**5 Phân tích biểu mẫu** **19**
- 5.1 Parse đơn thành danh sách trường . . . 19
- 5.2 Chuyển danh sách sang JSON . . . 19
- 5.3 Đưa parser vào workflow . . . 20
- 5.4 Một truy vấn cho mỗi trường . . . 21
- 5.5 Chạy thử và quan sát . . . 23

**6 Con người trong vòng lặp** **24**
6.1 Hai sự kiện đặc biệt . . . 24
6.2 Tách bước . . . 24
6.3 Tạm dừng chờ phản hồi . . . 26
6.4 Lồng phản hồi từ bên ngoài . . . 27
6.5 Thực sự dùng phản hồi . . . 28

**7 Dùng giọng nói** **29**
7.1 Nhìn lại workflow . . . 29
7.2 Phiên âm bằng Whisper . . . 29
7.3 Bắt âm thanh với Gradio . . . 30
7.4 Lớp xử lý transcription . . . 30
7.5 Dùng giọng nói thay bàn phím . . . 31
7.6 Tài liệu tham khảo thêm . . . 32

# Chương 1

# Tổng quan

Tài liệu này là bộ ghi chú học tập về **quy trình xử lý tài liệu bằng tác tử** (Agentic Document Workflows – ADW), biên soạn dựa trên khoá *Event-Driven Agentic Document Workflows* của DeepLearning.AI hợp tác với LlamaIndex. Tài liệu đi theo hướng **high-level, trực quan và thực chiến**: ưu tiên hiểu ý tưởng và biết cách dùng thư viện `llama-index`, thay vì tự cài đặt từng thành phần từ đầu.

Xuyên suốt khoá học, ta xây một **tác tử điền đơn**: cho một bản CV (resume) và một mẫu đơn xin việc (job application form), tác tử sẽ tự đọc đơn, biến các ô trống thành câu hỏi, dùng RAG để tra thông tin từ CV, rồi điền đơn và cho phép người dùng góp ý chỉnh sửa – bằng văn bản hoặc bằng giọng nói.

## 1.1 Triết lý trình bày

- **Trực giác trước, chi tiết sau** – mỗi khái niệm được giải thích bằng ví dụ đời thường và sơ đồ. Thay vì công thức toán, tài liệu mô tả bằng lời: nói rõ cái gì vào, cái gì ra, và vì sao.
- **Thực chiến bằng thư viện** – code ví dụ dùng hệ sinh thái LlamaIndex (gói `llama_index`, lớp `VectorStoreIndex`, trình parse `LlamaParse`) và các mô hình của OpenAI (`gpt-4o-mini` để sinh, `text-embedding-3-small` để nhúng, `whisper-1` để phiên âm). Mục tiêu là chạy được việc thật.
- **Sơ đồ hoá** – luồng sự kiện trong workflow và quan hệ giữa các bước đều được minh hoạ bằng sơ đồ để dễ nhớ và dễ tra cứu.

## 1.2 Cách đọc tài liệu

Mỗi chương gồm ba lớp nội dung đan xen:

- **Lý thuyết** – giải thích bằng ví dụ và sơ đồ, hạn chế ký hiệu toán.
- **Code** – ví dụ Python đặt trong khung, có chú thích tiếng Việt, kèm giải thích từng bước dùng để làm gì.
- **Áp dụng** – lưu ý khi đưa vào dự án thực tế, điểm cần cẩn trọng.

## 1.3 Nội dung các chương

| Chương | Nội dung chính |
|---|---|
| Ch. 2: Khái niệm ADW | RAG là gì và vì sao cần nó, giới hạn của RAG, agent và workflow, định nghĩa Agentic Document Workflow. |
| Ch. 3: Xây dựng workflow | workflow trong LlamaIndex, bước và sự kiện, rẽ nhánh, vòng lặp, chạy song song, gom sự kiện, truyền sự kiện theo luồng (streaming). |
| Ch. 4: Thêm RAG | dùng LlamaParse đọc CV, dựng vector store, query engine, biến RAG thành công cụ cho agent, đóng gói thành workflow. |
| Ch. 5: Phân tích biểu mẫu | parse đơn thành danh sách trường, chuyển sang JSON, phát một truy vấn cho mỗi trường rồi tổng hợp câu trả lời. |
| Ch. 6: Con người trong vòng lặp | tách bước, sự kiện yêu cầu nhập liệu, tạm dừng chờ phản hồi, lồng góp ý của người dùng vào kết quả. |
| Ch. 7: Dùng giọng nói | thêm năng lực đa phương thức, phiên âm bằng Whisper, bắt âm thanh với Gradio, góp ý bằng lời nói. |

## 1.4 Nguồn và quy ước

Nội dung được biên soạn và diễn giải lại từ khoá học *Event-Driven Agentic Document Workflows* của DeepLearning.AI và LlamaIndex. Các đoạn mô tả đã được viết lại bằng tiếng Việt cho phù hợp mục đích học tập; phần code giữ nguyên cú pháp cần thiết để chạy được. Mọi tên hàm, tên lớp và kết quả minh hoạ (ví dụ tên ứng viên trong CV giả lập) đều lấy trực tiếp từ ví dụ trong tài liệu gốc.

# Chương 2

# Khái niệm Agentic Document Workflows

Chương này trình bày các khái niệm nền tảng đứng sau quy trình xử lý tài liệu bằng tác tử: **RAG**, **agent** và **workflow**. Hiểu được ba thứ này, ta sẽ thấy ADW đơn giản chỉ là cách ghép chúng lại để giải những bài toán thực tế.

## 2.1 RAG là gì

RAG viết tắt của *Retrieval Augmented Generation* – sinh văn bản có tăng cường bằng truy xuất. Nó ra đời để vá một giới hạn cốt lõi của LLM: mô hình được huấn luyện trên khối dữ liệu khổng lồ, nhưng *không* được huấn luyện trên dữ liệu riêng của bạn. Khi giải một bài toán thật, bạn thường cần xử lý dữ liệu riêng tư chứ không chỉ kiến thức tổng quát.

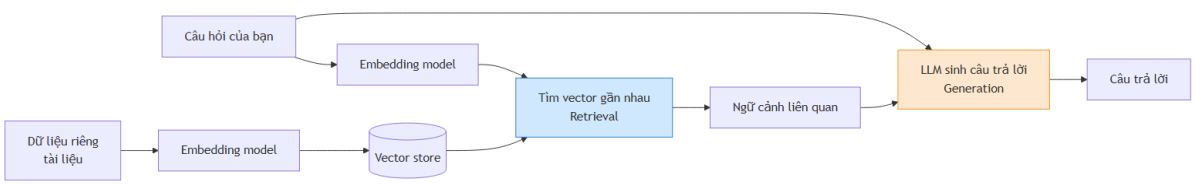
Muốn LLM trả lời về dữ liệu của bạn thì phải đưa dữ liệu đó cho nó. Nhưng ở đây lại gặp giới hạn thứ hai: **cửa sổ ngữ cảnh** (context window). Mỗi lần LLM chỉ nhận được một lượng dữ liệu hữu hạn; ngay cả mô hình mạnh nhất cũng chỉ xử lý được cỡ một triệu token mỗi lần, trong khi tổ chức của bạn có thể có hàng chục đến hàng trăm triệu token. Vì vậy phải chọn lọc: đưa cho LLM phần dữ liệu *liên quan nhất*.

### 2.1.1 Embedding và vector space

Chính công nghệ tạo ra LLM cũng tạo ra **embedding model**. Mô hình này nhận một chuỗi dữ liệu và mã hoá thành một mảng số gọi là **vector**. Tập hợp mọi vector khả dĩ là **không gian vector** (vector space). Điều kỳ diệu là vector mã hoá *ý nghĩa* của dữ liệu: hai đoạn nói về những điều tương tự sẽ nằm gần nhau trong không gian vector.

Nhờ đó, luồng RAG hoạt động như Hình 2.1:

1. Mã hoá dữ liệu riêng thành vector và lưu vào **vector store**.
2. Khi có câu hỏi, chạy câu hỏi qua *cùng* embedding model để ra một vector.
3. Tìm trong cơ sở dữ liệu những vector nằm gần vector câu hỏi – đó là bước **retrieval** (truy xuất).
4. Lấy câu hỏi cùng dữ liệu liên quan (gọi là **context**) đưa cho một LLM và yêu cầu trả lời dựa trên context – đó là bước **generation** (sinh).




Hình 2.1: Luồng RAG: truy xuất dữ liệu liên quan rồi sinh câu trả lời.

## 2.2 Giới hạn của RAG

RAG rất mạnh nhưng có điểm yếu: **câu hỏi phức tạp, nhiều phần**. Điều này dễ hiểu vì RAG dựa trên tìm kiếm. Một câu hỏi gồm nhiều phần sẽ khiến RAG tìm nhiều thứ cùng lúc, nên kết quả kém tập trung: bạn nhận về rất nhiều mẩu dữ liệu nhưng có thể vẫn thiếu thông tin cần thiết.

Giải pháp là **tách câu hỏi phức tạp thành nhiều câu hỏi đơn giản**. Mỗi câu hỏi đơn sẽ cho kết quả tìm kiếm tập trung và đầy đủ hơn. Đây đúng là việc LLM giỏi: nhìn một câu hỏi rối rắm và chia nhỏ ra, rồi ở đầu bên kia, gom các câu trả lời đơn lại thành một câu trả lời mạch lạc. Và cách để làm việc này là xây một **agent**.

## 2.3 Agent và workflow

### 2.3.1 Agent

*Agent* là một thuật ngữ khá mơ hồ. Ở LlamaIndex, agent được hiểu là một **phần mềm bán tự chủ**: được giao *công cụ* và *mục tiêu*, nó tự tìm cách giải quyết vấn đề mà không cần được chỉ dẫn từng bước. Điều này khác hẳn lập trình truyền thống, nơi mọi bước đều được định nghĩa chính xác.

### 2.3.2 Workflow

Cách xây agent trong LlamaIndex là dùng **workflow** – khối xây dựng của hệ thống tác tử. Workflow là một hệ thống **dựa trên sự kiện** (event-based): bạn định nghĩa một chuỗi bước nối với nhau bằng sự kiện, và dữ liệu truyền từ bước này sang bước khác qua các sự kiện đó. Bạn có thể tạo workflow khá phức tạp với rẽ nhánh, vòng lặp và chạy song song.

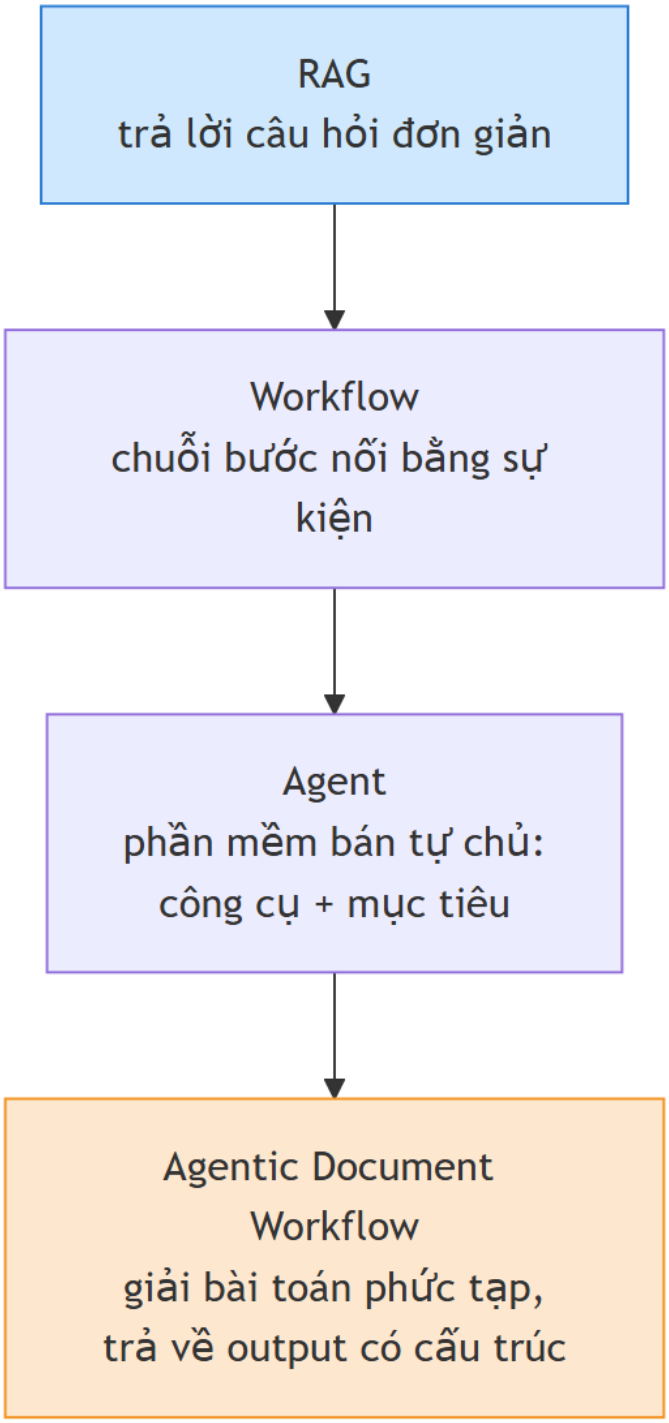
**Vì sao chọn workflow?**

Một số framework agent *không* có cấu trúc nào, dễ dẫn tới kết quả hỗn loạn. Số khác dùng cách tiếp cận *đồ thị* (graph), khiến vòng lặp và các cấu trúc khác trở nên khó làm. Workflow nằm giữa hai thái cực: cho cấu trúc đủ chi tiết hoặc đủ lỏng tuỳ nhu cầu, nên dễ dùng mà vẫn linh hoạt.

## 2.4 Agentic Document Workflows

Ghép ba mảnh RAG, workflow và agent lại, ta có **Agentic Document Workflows** (ADW). Đây là cách xây phần mềm giải bài toán kinh doanh thật bằng cách áp workflow tác tử vào bài toán và lắp nó vào một hệ thống lớn hơn (Hình 2.2).

ADW xây trên sức mạnh của RAG. Nhưng khác với RAG – vốn chủ yếu dành cho câu hỏi đơn giản – ADW xử lý **bài toán phức tạp** và cho ra **output có cấu trúc, cụ thể**, không chỉ là một đoạn trả lời bằng tiếng Anh thuần.




Hình 2.2: ADW xếp chồng trên RAG, workflow và agent.

Vài ví dụ ứng dụng:

- **Rà soát hợp đồng**: agent parse hợp đồng, trích các điều khoản chính, đối chiếu với kho quy định bằng RAG, rồi sinh một bản tóm tắt tuân thủ.
- **Chuẩn hoá hoá đơn**: agent trích mô tả mặt hàng từ hoá đơn, khớp với mã sản phẩm gần nhất trong danh mục bằng RAG, rồi gắn thông tin chuẩn hoá vào hoá đơn.
- **Điền đơn từ CV** (bài toán của khoá này): agent dùng CV để điền một mẫu đơn xin việc.

# Chương 3

# Xây dựng workflow

Chương này xây dần các workflow từ đơn giản tới phức tạp để nắm vững khái niệm cốt lõi: bắt đầu bằng một chuỗi bước phát sự kiện, rồi thêm **rẽ nhánh** và **vòng lặp**, thực hiện **chạy song song**, và cuối cùng **gom sự kiện** khác loại tại một bước. Dưới nắp capô, workflow chỉ là các lớp Python thông thường.

## 3.1 Workflow một bước

Workflow được định nghĩa là một chuỗi **bước** (step), mỗi bước nhận một số loại sự kiện và phát ra một số loại sự kiện. Ta import phần lõi của workflow từ `llama_index.core.workflow`, trong đó có hai sự kiện đặc biệt là `StartEvent` và `StopEvent`.

```
from llama_index.core.workflow import (
    StartEvent,
    StopEvent,
    Workflow,
    step,
    Context,
)

class MyWorkflow(Workflow):
    # khai báo một hàm là một step
    @step
    async def my_step(self, ev: StartEvent) -> StopEvent:
        # xử lý ở đây
        return StopEvent(result="Hello, world!")
```

Đây là một khai báo lớp bình thường, một decorator `@step`, và một hàm `async`. `StartEvent` được kích hoạt khi workflow bắt đầu; khi `StopEvent` được phát ra, workflow tự động dừng. Từ khoá `async` định nghĩa hàm bất đồng bộ – có thể tạm dừng và tiếp tục, cho phép việc khác chạy xen vào (hữu ích khi chạy song song ở phần sau).

```
basic_workflow = MyWorkflow(timeout=10, verbose=False)
result = await basic_workflow.run()
print(result)
```

Workflow mặc định bất đồng bộ, nên dùng `await` để lấy kết quả của `run()`. Cách này chạy tốt trong môi trường notebook; trong một script Python thuần, bạn cần `import asyncio` và bọc code trong một hàm async rồi gọi `asyncio.run(...)`.

**Trình trực quan hoá có sẵn**

Workflow có hàm `draw_all_possible_flows(workflow, filename)` (import từ `llama_index.utils.workflow`) xuất ra một file HTML chứa sơ đồ tương tác của workflow. Trong notebook, khoá học dùng một hàm trợ giúp `extract_html_content` rồi `display(HTML(...))` để hiển thị trực tiếp sơ đồ ra ô kết quả; trên máy cá nhân chỉ cần mở file HTML trong trình duyệt. Mẹo hay: có thể truyền thẳng *lớp* workflow (chưa khởi tạo) cho hàm trực quan hoá, không cần tạo đối tượng trước.

## 3.2 Workflow nhiều bước

Nhiều bước được tạo bằng cách định nghĩa **sự kiện tuỳ biến**: một bước phát ra sự kiện, sự kiện đó kích hoạt bước khác. Các lớp sự kiện có thể mang tên và thuộc tính bất kỳ, nhưng phải kế thừa `Event`.

```
from llama_index.core.workflow import Event

class FirstEvent(Event):
    first_output: str

class SecondEvent(Event):
    second_output: str

class MyWorkflow(Workflow):
    @step
    async def step_one(self, ev: StartEvent) -> FirstEvent:
        print(ev.first_input)
        return FirstEvent(first_output="First step complete.")

    @step
    async def step_two(self, ev: FirstEvent) -> SecondEvent:
        print(ev.first_output)
        return SecondEvent(second_output="Second step complete.")

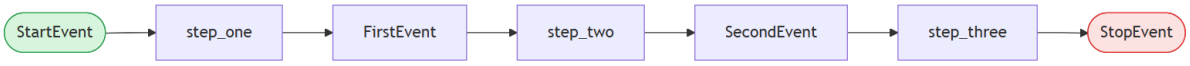
    @step
    async def step_three(self, ev: SecondEvent) -> StopEvent:
        print(ev.second_output)
        return StopEvent(result="Workflow complete.")
```

Bạn định nghĩa workflow bằng cách khai báo mỗi bước *nhận* sự kiện gì và *phát* sự kiện gì – đó chính là cách dữ liệu chảy quanh workflow (Hình 3.1). Chạy workflow này ta truyền thuộc tính cho `StartEvent` qua tham số của `run`:

```
workflow = MyWorkflow(timeout=10, verbose=False)
result = await workflow.run(first_input="Start the workflow.")
print(result)
```

**StartEvent và StopEvent**

Với `StartEvent`, bạn tự định nghĩa các thuộc tính và truyền giá trị vào khi gọi `run`. Với `StopEvent`, mặc định nó chỉ có một thuộc tính `result`; muốn tuỳ biến giá trị trả về, hãy tạo một lớp kế thừa `StopEvent`.




Hình 3.1: Workflow ba bước nối với nhau bằng sự kiện tuỳ biến.

## 3.3 Vòng lặp

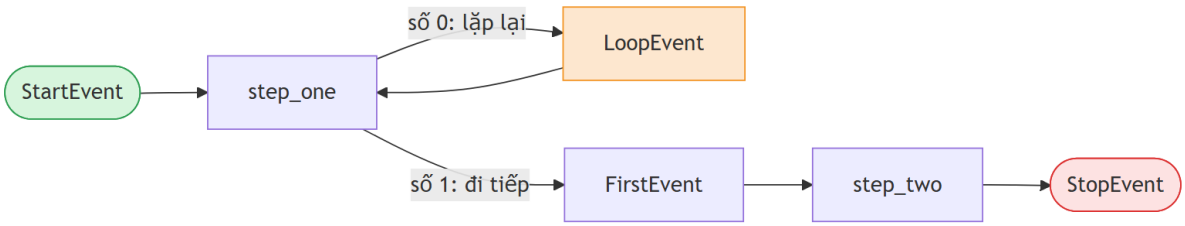
Một tính năng cốt lõi của workflow là rẽ nhánh và lặp đơn giản, linh hoạt hơn so với cách tiếp cận đồ thị. Để tạo vòng lặp, ta thêm một `LoopEvent` thường (không phải sự kiện đặc biệt).

```
import random

class LoopEvent(Event):
    loop_output: str

class MyWorkflow(Workflow):
    @step
    async def step_one(self, ev: StartEvent | LoopEvent) -> FirstEvent | LoopEvent:
        if random.randint(0, 1) == 0:
            print("Bad thing happened")
            return LoopEvent(loop_output="Back to step one.")
        else:
            print("Good thing happened")
            return FirstEvent(first_output="First step complete.")
```

`step_one` giờ nhận *hoặc* `StartEvent` *hoặc* `LoopEvent`, và phát *hoặc* `FirstEvent` *hoặc* `LoopEvent` (Hình 3.2). Vòng lặp có thể đặt ở bất cứ đâu – không nhất thiết lặp về chính bước đã bắt đầu.




Hình 3.2: Dùng `LoopEvent` để lặp lại một bước.

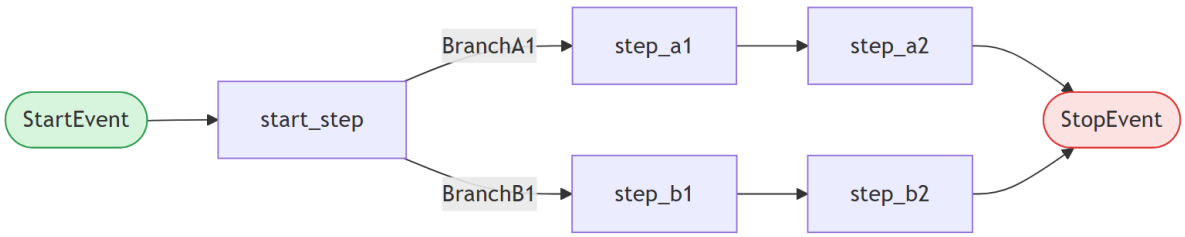
## 3.4 Rẽ nhánh

Cùng cấu trúc cho phép lặp cũng cho phép rẽ nhánh. Workflow dưới đây chạy một trong hai nhánh tuỳ theo một quyết định sớm (Hình 3.3).

```
class BranchA1Event(Event): payload: str
class BranchB1Event(Event): payload: str
# ... BranchA2Event, BranchB2Event tương tự

class BranchWorkflow(Workflow):
    @step
    async def start(self, ev: StartEvent) -> BranchA1Event | BranchB1Event:
        if random.randint(0, 1) == 0:
            return BranchA1Event(payload="Đi nhánh A")
        else:
            return BranchB1Event(payload="Đi nhánh B")
```

```
    # step_a1 -> step_a2 -> StopEvent ; step_b1 -> step_b2 -> StopEvent
```




Hình 3.3: Rẽ nhánh: hai đường đi riêng, cùng kết thúc ở `StopEvent`.

## 3.5 Chạy song song

Dạng điều khiển luồng cuối cùng là **chạy song song**, cho phép thực thi hiệu quả các tác vụ dài song song và gom chúng lại khi cần (kiểu MapReduce). Ở đây ta dùng **context object** – một dạng bộ nhớ chia sẻ cho mọi bước trong workflow. Khai báo `ctx: Context` làm tham số của bước thì nó tự được truyền vào.

Thay vì *trả về* một sự kiện, ta dùng `ctx.send_event(...)` để phát *nhiều* sự kiện song song.

```
from workflows import Context

class StepTwoEvent(Event):
    query: str

class ParallelFlow(Workflow):
    @step
    async def start(self, ctx: Context, ev: StartEvent) -> None:
        ctx.send_event(StepTwoEvent(query="Query 1"))
        ctx.send_event(StepTwoEvent(query="Query 2"))
        ctx.send_event(StepTwoEvent(query="Query 3"))

    @step(num_workers=4)
    async def step_two(self, ctx: Context, ev: StepTwoEvent) -> StopEvent:
        await asyncio.sleep(random.randint(1, 5))
        return StopEvent(result=ev.query)
```

Tham số `num_workers` đặt số bước `step_two` chạy song song (ở đây 4, cũng là mặc định). Mỗi `step_two` chờ ngẫu nhiên 1–5 giây rồi trả về `StopEvent`. Vì `StopEvent` *đầu tiên* phát ra sẽ dừng workflow, nên mỗi lần chạy có thể cho ra một query khác thắng cuộc.

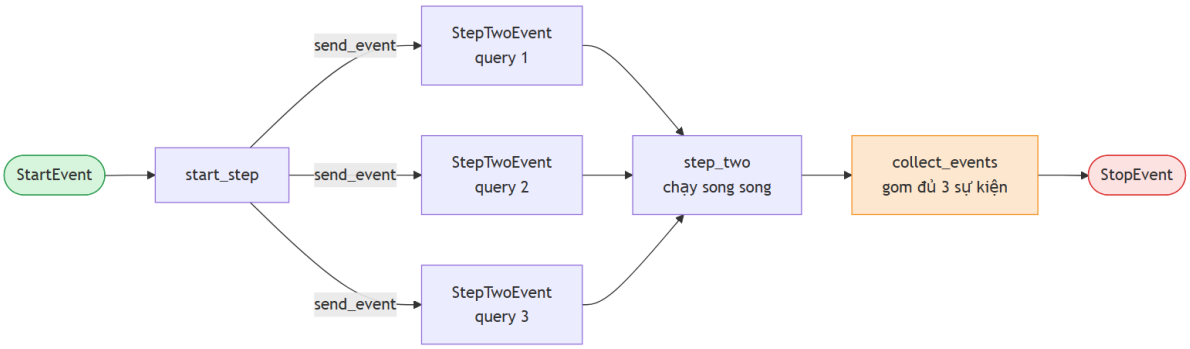
### 3.5.1 Gom nhiều sự kiện cùng loại

Muốn lấy output của *tất cả* chứ không chỉ cái đầu tiên, dùng `ctx.collect_events(...)`. Mỗi lần chạy, nếu chưa đủ số sự kiện cần, nó trả về `None`; khi đã đủ, nó trả về một mảng (Hình 3.4).

```
class ConcurrentFlow(Workflow):
    @step
    async def start(self, ctx: Context, ev: StartEvent) -> StepTwoEvent:
        ctx.send_event(StepTwoEvent(query="Query 1"))
        ctx.send_event(StepTwoEvent(query="Query 2"))
        ctx.send_event(StepTwoEvent(query="Query 3"))
```

```
    @step(num_workers=4)
    async def step_two(self, ctx: Context, ev: StepTwoEvent) -> StepThreeEvent:
        await asyncio.sleep(random.randint(1, 5))
        return StepThreeEvent(result=ev.query)

    @step
    async def step_three(self, ctx: Context, ev: StepThreeEvent) -> StopEvent:
        # chờ cho tới khi nhận đủ 3 sự kiện
        result = ctx.collect_events(ev, [StepThreeEvent] * 3)
        if result is None:
            print("Not all events received yet.")
            return None            # chưa gom đủ sự kiện
        print(result)
        return StopEvent(result="Done")
```




Hình 3.4: Chạy song song rồi `collect_events` gom đủ ba sự kiện.

Các sự kiện được trả về theo *thứ tự nhận được*, nên mỗi lần chạy thứ tự có thể khác nhau vì các bước hoàn tất ở những thời điểm ngẫu nhiên.

**Map-reduce không cần biết trước số bước**

Cách điều khiển luồng này cho phép làm các tác vụ kiểu **map-reduce**. Khi không biết trước có bao nhiêu bước song song, hãy lưu con số đó vào `Context` bằng `await ctx.set("num_events", n)`, rồi ở bước gom lại đọc ra bằng `await ctx.get("num_events")` và truyền vào `collect_events`. Ta sẽ dùng đúng mẫu này ở chương điền đơn.

### 3.5.2 Gom sự kiện khác loại

Có thể dùng rẽ nhánh để phát *nhiều loại* sự kiện khác nhau rồi chờ tất cả. Khi đó `collect_events` nhận một danh sách các loại, ví dụ `[StepCEvent, StepAEvent, StepBEvent]`. **Thứ tự trong danh sách này có ý nghĩa**: kết quả trả về sẽ theo đúng thứ tự bạn khai báo, nhờ đó bạn biết chắc phần tử nào ứng với sự kiện nào.

## 3.6 Truyền sự kiện theo luồng (streaming)

Trong thực tế, agent có thể chạy khá lâu. Bắt người dùng chạy rồi ngồi chờ mà không biết gì là trải nghiệm tệ. Tốt hơn là cho họ thấy tiến trình theo thời gian thực. Workflow cho phép **stream sự kiện** về người dùng bằng `ctx.write_event_to_stream(...)`.

Đây cũng là lần đầu dùng LLM thật trong chương. Khi yêu cầu LLM sinh phản hồi dạng streaming, nó gửi về từng *chunk* ngay khi có; mỗi chunk là một **delta**. Ta bọc delta vào một `TextEvent` và đẩy vào luồng của workflow để echo ra cho người dùng.

```
from llama_index.llms.openai import OpenAI

class MyWorkflow(Workflow):
    @step
    async def step_one(self, ctx: Context, ev: StartEvent) -> FirstEvent:
        ctx.write_event_to_stream(ProgressEvent(msg="Đang chạy bước 1"))
        return FirstEvent(first_output="Bước 1 xong.")

    @step
    async def step_two(self, ctx: Context, ev: FirstEvent) -> SecondEvent:
        llm = OpenAI(model="gpt-4o-mini", api_key=api_key)
        generator = await llm.astream_complete(
            "Please give me the first 50 words of Moby Dick, "
            "a book in the public domain."
        )
        async for response in generator:
            # cho phép workflow stream từng mẩu phản hồi
            ctx.write_event_to_stream(TextEvent(delta=response.delta))
        return SecondEvent(
            second_output="Second step complete, full response attached",
            response=str(response),
        )

    @step
    async def step_three(self, ctx: Context, ev: SecondEvent) -> StopEvent:
        ctx.write_event_to_stream(ProgressEvent(msg="Step three is happening"))
        return StopEvent(result="Workflow complete.")
```

Lưu ý `astream_complete` (chữ *a* nghĩa là bất đồng bộ) khác với `stream_complete` vốn chặn (blocking). Khi chạy, thay vì `await workflow.run()`, ta gọi `workflow.run()` lấy một handler rồi lặp qua `handler.stream_events()` để lọc đúng loại sự kiện cần in ra:

```
workflow = MyWorkflow(timeout=30, verbose=True)
handler = workflow.run(first_input="Start the workflow.")

async for ev in handler.stream_events():
    if isinstance(ev, ProgressEvent):
        print(ev.msg)
    if isinstance(ev, TextEvent):
        print(ev.delta, end="")

final_result = await handler
print("Final result = ", final_result)
```

Kết quả từ LLM xuất hiện theo từng chunk thay vì hiện ra cùng lúc – đúng tinh thần streaming. Chương sau sẽ thêm RAG vào workflow.

# Chương 4

# Thêm RAG

Chương này dựng năng lực RAG cho agent: dùng **LlamaParse** đọc một bản CV, nạp thông tin trích được vào **vector store**, rồi chạy truy vấn cơ bản để kiểm thử. Cuối cùng, ta biến RAG thành một **công cụ** mà agent có thể dùng, và gói tất cả vào một workflow.

## 4.1 Parse CV bằng LlamaParse

**LlamaParse** là trình parse tài liệu nâng cao, đọc được PDF, Word, PowerPoint, Excel... và rất giỏi lấy thông tin từ PDF phức tạp ra dạng mà LLM dễ hiểu. Một điểm hay là có thể báo cho nó biết đang đọc loại tài liệu gì để parse thông minh hơn.

**Chuẩn bị môi trường**

Notebook cần `nest_asyncio` (gọi `nest_asyncio.apply()`) để chạy được async lồng nhau – cho phép lồng các event loop của asyncio vào nhau. Ngoài khoá OpenAI dùng cho embedding, ta cần thêm **Llama Cloud API key** (lấy miễn phí tại `https://cloud.llamaindex.ai`) vì LlamaParse chạy trên đám mây.

```
import os
from llama_parse import LlamaParse

documents = LlamaParse(
    api_key=llama_cloud_api_key,
    base_url=os.getenv("LLAMA_CLOUD_BASE_URL"),
    result_type="markdown",
    content_guideline_instruction=(
        "This is a resume, gather related facts together and format "
        "it as bullet points with headers"
    ),
).load_data("data/fake_resume.pdf")

print(documents[2].text)
```

Ta yêu cầu `result_type="markdown"` và cho một `content_guideline_ instruction` nói rằng đây là CV, hãy gom các dữ kiện liên quan và trình bày thành gạch đầu dòng có tiêu đề. `load_data` trả về một mảng `Document`; in ra một phần tử ta thấy nó đã tách các dự án, công ty và việc đã làm theo đúng định dạng markdown.

**Document mang theo gì?**

Mỗi `Document` ngoài phần văn bản còn chứa `metadata` (từ điển chú thích có thể gắn kèm) và `relationships` (quan hệ tới các `Document` khác). Chính mảng `Document` này sẽ được đưa vào `VectorStoreIndex`.

## 4.2 Dựng vector store và query engine

Có tài liệu rồi, ta đưa vào `VectorStoreIndex`. Index sẽ **embed** văn bản – biến thành vector tìm kiếm được – qua một embedding API của OpenAI (lý do cần khoá OpenAI).

```
from llama_index.core import VectorStoreIndex
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI

index = VectorStoreIndex.from_documents(
    documents,
    embed_model=OpenAIEmbedding(model_name="text-embedding-3-small",
                                api_key=openai_api_key),
)

llm = OpenAI(model="gpt-4o-mini")
query_engine = index.as_query_engine(llm=llm, similarity_top_k=5)

response = query_engine.query(
    "What is this person's name and what was their most recent job?"
)
print(response)
```

Ta dùng mô hình embedding `text-embedding-3-small` và LLM `gpt-4o-mini` (nhanh và rẻ). Khi gọi `as_query_engine`, tham số `similarity_top_k=5` yêu cầu query engine chỉ trả về 5 mẩu context liên quan nhất thay vì tất cả. Chạy thử với câu hỏi cơ bản, ta nhận được: ứng viên giả lập tên **Sarah Chen**, công việc gần nhất là **Senior Full Stack Developer tại TechFlow Solutions**.

### 4.2.1 Lưu và nạp lại vector store

Vector store dễ dàng lưu xuống đĩa để dùng lại:

```
from llama_index.core import StorageContext, load_index_from_storage

storage_dir = "./storage"
# Lưu
index.storage_context.persist(persist_dir=storage_dir)

# Nạp lại
if os.path.exists(storage_dir):
    storage_context = StorageContext.from_defaults(persist_dir=storage_dir)
    restored_index = load_index_from_storage(storage_context)
else:
    print("Index not found on disk.")

response = restored_index.as_query_engine().query(
    "What is this person's name and what was their most recent job?")
print(response)
```

Ta kiểm tra: nếu thư mục lưu đã tồn tại thì nạp lại; nếu chưa thì báo không tìm thấy. Nhờ đã `persist` trước đó, lần sau index được nạp lại tức thì mà không phải parse lại tài liệu. Trong môi trường production, thường bạn sẽ dùng một vector store được hosting thay vì lưu xuống đĩa cục bộ như ví dụ này.

## 4.3 Biến RAG thành công cụ cho agent

Để tiến tới hệ thống tác tử, ta bọc pipeline RAG thành một **tool**. Cần hai lớp: `FunctionTool` và một agent gọi hàm (function-calling agent). Trước hết, viết một hàm Python thường thực hiện truy vấn RAG, đặt *tên gợi nhớ*, chú thích kiểu vào/ra, và viết **docstring** mô tả rõ chức năng.

**Vì sao docstring quan trọng**

Framework sẽ đưa toàn bộ *metadata* của hàm – tên, kiểu dữ liệu và docstring – cho LLM. LLM dựa vào đó để quyết định công cụ này có hữu ích không và dùng nó thế nào. Viết tên và docstring cẩu thả nghĩa là agent sẽ dùng công cụ sai.

```
from llama_index.core.tools import FunctionTool
from llama_index.core.agent import FunctionCallingAgent

def query_resume(q: str) -> str:
    """Answers questions about a specific resume."""
    # dùng lại query_engine đã tạo ở trên
    response = query_engine.query(
        f"This is a question about the specific resume we have in our "
        f"database: {q}"
    )
    return response.response

resume_tool = FunctionTool.from_defaults(fn=query_resume)

agent = FunctionCallingAgent.from_tools(
    tools=[resume_tool],
    llm=llm,
    verbose=True,
)

response = agent.chat(
    "How many years of experience does the applicant have?"
)
print(response)
```

`FunctionCallingAgent` (tạo qua `.from_tools`) là một loại agent mạnh và hiệu quả, hợp với OpenAI. Khi `verbose=True`, output khá ồn: nó cho biết đang thêm câu hỏi vào bộ nhớ, chọn công cụ nào, gọi `query_resume` với tham số gì, hàm trả về gì, và cuối cùng LLM trả lời ra sao.

## 4.4 Đóng gói thành workflow

Cuối cùng, gói pipeline RAG và agent gọn gàng vào một workflow để các chương sau mở rộng tiếp (Hình 4.1). Workflow này không dùng lại gì đã tạo ở trên – nó bắt đầu từ đầu.

```
from llama_index.core.workflow import (
    StartEvent, StopEvent, Workflow, step, Event, Context,
)
```

```
class QueryEvent(Event):
    query: str

class RAGWorkflow(Workflow):
    storage_dir = "./storage"
    llm: OpenAI
    query_engine: VectorStoreIndex

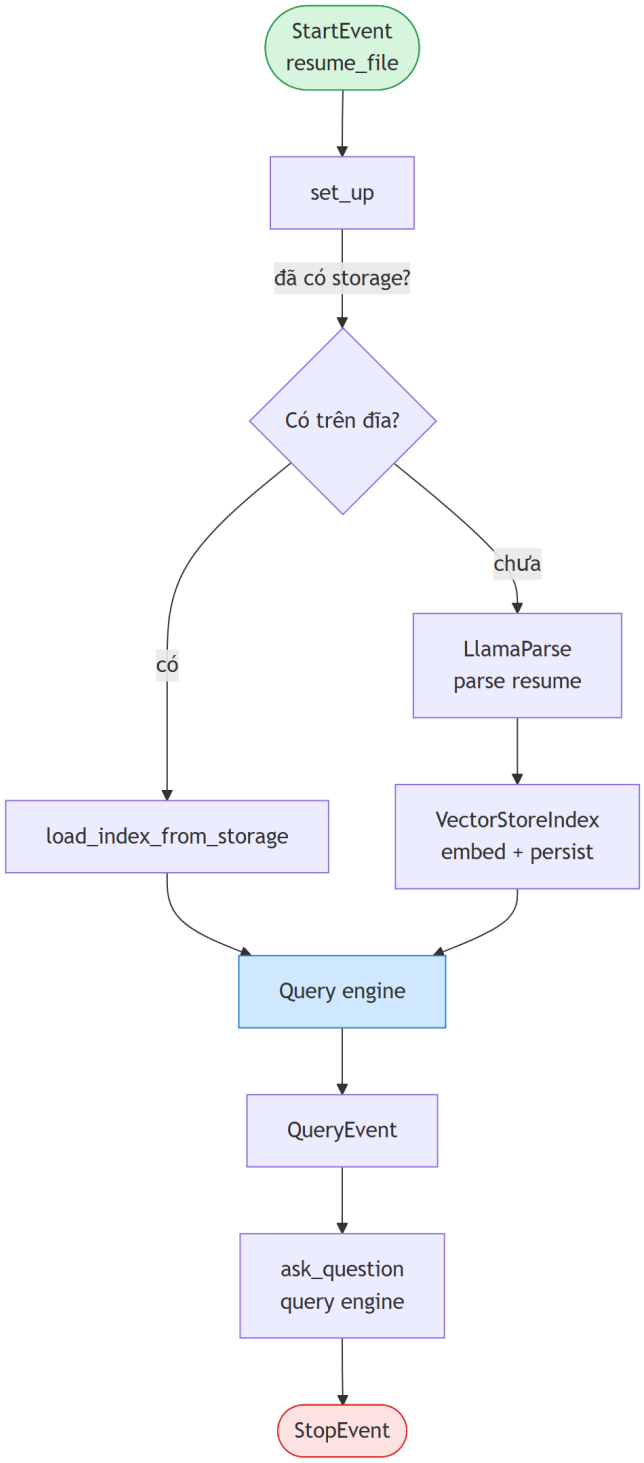
    @step
    async def set_up(self, ctx: Context, ev: StartEvent) -> QueryEvent:
        if not ev.resume_file:
            raise ValueError("No resume file provided")

        self.llm = OpenAI(model="gpt-4o-mini")

        if os.path.exists(self.storage_dir):
            # Đã có: nạp lại từ đĩa
            storage_context = StorageContext.from_defaults(
                persist_dir=self.storage_dir)
            index = load_index_from_storage(storage_context)
        else:
            # Chưa có: parse + nạp tài liệu
            documents = LlamaParse(
                result_type="markdown",
                content_guideline_instruction="This is a resume, gather "
                    "related facts together and format it as bullet points "
                    "with headers",
            ).load_data(ev.resume_file)
            index = VectorStoreIndex.from_documents(
                documents,
                embed_model=OpenAIEmbedding(model_name="text-embedding-3-small"),
            )
            index.storage_context.persist(persist_dir=self.storage_dir)

        self.query_engine = index.as_query_engine(
            llm=self.llm, similarity_top_k=5)
        return QueryEvent(query=ev.query)

    @step
    async def ask_question(self, ctx: Context, ev: QueryEvent) -> StopEvent:
        response = self.query_engine.query(
            f"This is a question about the specific resume we have in our "
            f"database: {ev.query}"
        )
        return StopEvent(result=response.response)
```




Hình 4.1: RAGWorkflow: thiết lập index (nạp lại hoặc parse mới) rồi trả lời truy vấn.

```
w = RAGWorkflow(timeout=120, verbose=False)
result = await w.run(
    resume_file="./data/fake_resume.pdf",
    query="Where is the first place the applicant worked?",
)
print(result)
```

Workflow nhận hai tham số: vị trí file CV để parse và câu hỏi về ứng viên. Vì đã persist từ trước nên

nó trả lời rất nhanh, không cần parse lại tài liệu.

**Một lỗi nhỏ cần suy nghĩ**

Có một lỗi tinh vi: nếu chạy lần hai với một CV *khác*, code sẽ thấy storage cũ trên đĩa và dùng lại, nên *không* parse CV mới. Chưa cần sửa ngay, nhưng hãy nghĩ cách khắc phục (ví dụ gắn storage theo từng CV).

Ở đây chưa có gì mới ngoài việc đóng gói gọn gàng. Chương sau sẽ giao cho agent nhiệm vụ phức tạp hơn: đọc và điền một mẫu đơn.

# Chương 5

# Phân tích biểu mẫu

Chương này giao cho agent nhiệm vụ phức tạp hơn: **parse mẫu đơn xin việc**, biến đơn đã parse thành **một loạt câu hỏi đơn giản**, rồi dùng các câu hỏi đó truy vấn pipeline RAG để điền đơn. Ta lại dùng LlamaParse, nhưng với chỉ dẫn nâng cao hơn.

## 5.1 Parse đơn thành danh sách trường

Lần trước, LlamaParse được dùng để parse một bản CV. Lần này chỉ dẫn phức tạp hơn: đọc mẫu đơn và chuyển thành danh sách các **ô cần điền**.

```
import os, json
from llama_parse import LlamaParse

parser = LlamaParse(
    api_key=llama_cloud_api_key,
    base_url=os.getenv("LLAMA_CLOUD_BASE_URL"),
    result_type="markdown",
    content_guideline_instruction=(
        "This is a job application form. Create a list of all the fields "
        "that need to be filled in."
    ),
    formatting_instruction=(
        "Return a bulleted list of the fields ONLY."
    ),
)
result = parser.load_data("data/fake_application_form.pdf")[0]
print(result.text)
```

Ta cho hai chỉ dẫn: `content_guideline_instruction` (đây là đơn xin việc, tạo danh sách các ô cần điền) và `formatting_instruction` (chỉ trả về danh sách gạch đầu dòng gồm các ô). Kết quả bám sát chỉ dẫn: một danh sách gọn gàng chỉ gồm các trường.

## 5.2 Chuyển danh sách sang JSON

Một việc LLM làm tốt là biến định dạng con người đọc được (danh sách gạch đầu dòng) thành định dạng máy đọc được. Ta nhờ LLM biến danh sách thành một đối tượng JSON có một mảng `fields`.

```
llm = OpenAI(model="gpt-4o-mini")

raw_json = llm.complete(
    f"""
```

```
    This is a parsed form.
    Convert it into a JSON object containing only the list
    of fields to be filled in, in the form {{ fields: [...] }}.
    <form>{result.text}</form>.
    Return JSON ONLY, no markdown."""
)
fields = json.loads(raw_json.text)["fields"]
for field in fields:
    print(field)
```

Giờ ta có một mảng các trường, sẵn sàng cho mọi xử lý lập trình phía sau.

## 5.3 Đưa parser vào workflow

Lấy workflow ở chương trước và thêm parser. Cần hai sự kiện mới: `ParseFormEvent` và `QueryEvent`. Bước `set_up` nay phát ra `ParseFormEvent` để kích hoạt bước mới `parse_form`:

```
class ParseFormEvent(Event):
    application_form: str

class QueryEvent(Event):
    query: str

class RAGWorkflow(Workflow):
    @step
    async def set_up(self, ctx: Context, ev: StartEvent) -> ParseFormEvent:
        if not ev.resume_file:
            raise ValueError("No resume file provided")
        if not ev.application_form:
            raise ValueError("No application form provided")
        # ... thiết lập index + query_engine như chương trước
        # không cần truyền query vào nữa, ta sẽ tự sinh query
        return ParseFormEvent(application_form=ev.application_form)

    @step
    async def parse_form(self, ctx: Context, ev: ParseFormEvent) -> QueryEvent:
        parser = LlamaParse(
            api_key=llama_cloud_api_key,
            base_url=os.getenv("LLAMA_CLOUD_BASE_URL"),
            result_type="markdown",
            content_guideline_instruction="This is a job application form. "
                "Create a list of all the fields that need to be filled in.",
            formatting_instruction="Return a bulleted list of the fields ONLY.",
        )
        result = parser.load_data(ev.application_form)[0]
        raw_json = self.llm.complete(
            f"...convert into JSON {{ fields: [...] }} ... {result.text}")
        fields = json.loads(raw_json.text)["fields"]
        for field in fields:
            print(field)
        # bản đầu chỉ in ra fields để kiểm thử rồi dừng tạm bằng StopEvent
        return StopEvent(result="Dummy event")
```

## 5.4 Một truy vấn cho mỗi trường

Giờ workflow đã biết cần những trường nào. Ở bước tiếp theo, ta phát **một `QueryEvent` cho mỗi trường** để chúng chạy song song (nhớ lại kỹ thuật chạy song song ở Chương 3). Thêm một `ResponseEvent` để chuyển kết quả truy vấn tới bước tổng hợp (Hình 5.1).

```
class QueryEvent(Event):
    query: str
    field: str

class ResponseEvent(Event):
    response: str

class RAGWorkflow(Workflow):
    @step
    async def parse_form(self, ctx: Context, ev: ParseFormEvent) -> QueryEvent:
        # ... parse + JSON như trên, có 'fields'
        # sinh một query cho mỗi trường rồi bắn đi
        for field in fields:
            ctx.send_event(QueryEvent(
                field=field,
                query=f"How would you answer this question about the "
                      f"candidate? {field}",
            ))
        # lưu số trường để biết phải chờ bao nhiêu
        await ctx.set("total_fields", len(fields))
        return

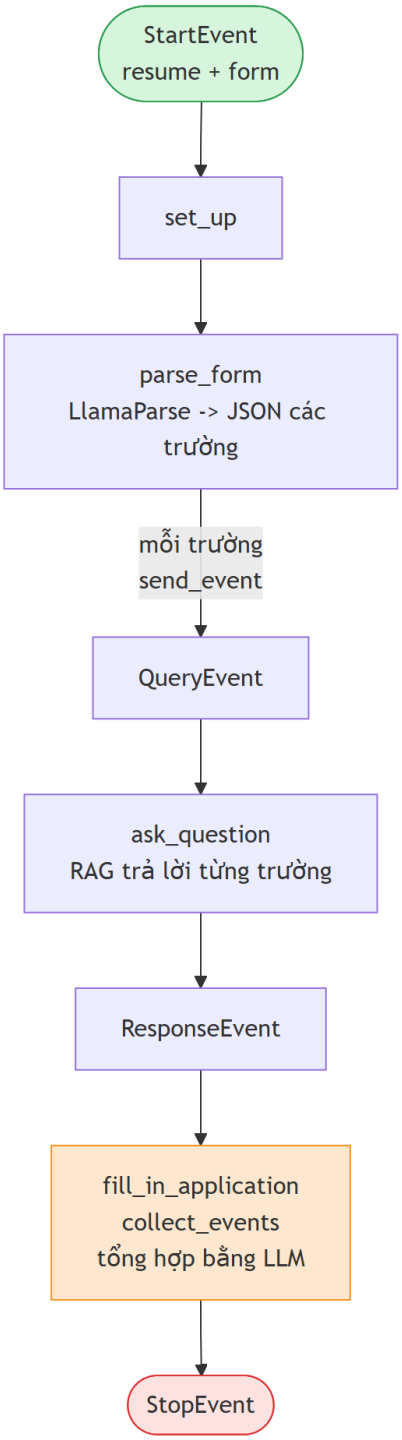
    @step
    async def ask_question(self, ctx: Context, ev: QueryEvent) -> ResponseEvent:
        response = self.query_engine.query(
            f"This is a question about the specific resume we have in our "
            f"database: {ev.query}"
        )
        return ResponseEvent(field=ev.field, response=response.response)

    @step
    async def fill_in_application(
        self, ctx: Context, ev: ResponseEvent
    ) -> StopEvent:
        total_fields = await ctx.get("total_fields")
        responses = ctx.collect_events(ev, [ResponseEvent] * total_fields)
        if responses is None:
            return None            # chưa gom đủ

        responseList = "\n".join(
            "Field: " + r.field + "\n" + "Response: " + r.response
            for r in responses)
        result = self.llm.complete(f"""
            You are given a list of fields in an application form and responses
            to questions about those fields from a resume. Combine the two into
            a list of fields and succinct, factual answers to fill in those
            fields.

            <responses>
            {responseList}
            </responses>
        """)
```

```
        return StopEvent(result=result)
```




Hình 5.1: Mỗi trường sinh một `QueryEvent`; `fill_in_application` gom đủ các `ResponseEvent` rồi tổng hợp bằng LLM.

Ta lưu `total_fields` vào `Context` bằng `await ctx.set(...)` để `collect_events` biết phải chờ bao nhiêu phản hồi – giá trị này dùng được xuyên các bước (đọc lại bằng `await ctx.get(...)`) mà không cần truyền qua sự kiện. Nhờ vậy workflow không cần biết trước có bao nhiêu trường, số lượng do form quyết định lúc chạy.

## 5.5 Chạy thử và quan sát

```
w = RAGWorkflow(timeout=120, verbose=False)
result = await w.run(
    resume_file="data/fake_resume.pdf",
    application_form="data/fake_application_form.pdf",
)
print(result)
```

Lần chạy này mất lâu hơn vì phải hỏi nhiều câu. Kết quả là một danh sách đánh số gồm tất cả các trường cùng câu trả lời. Nhìn chung tốt, nhưng vài trường có thể làm tốt hơn: ví dụ **portfolio** trong tài liệu gốc là một đường link, còn ở đây LLM lại liệt kê ra một danh mục. Chương sau sẽ thêm khả năng góp ý để sửa những chỗ như vậy.

# Chương 6

# Con người trong vòng lặp

Workflow ở chương trước nhận diện được mọi trường và sinh câu trả lời hợp lý, nhưng vài trường cần người chỉnh thêm. Chương này xây một workflow **human-in-the-loop**: người dùng có thể góp ý về câu trả lời, và agent sẽ trả lời lại khi cần.

**Nguyên tắc**

LLM tuyệt vời khi dùng để *hỗ trợ* con người chứ không phải *thay thế*. Trình điền đơn hiện tại làm tốt phần việc tìm ra các trường và điền đúng phần lớn, nhưng vài chỗ cần người giúp một chút.

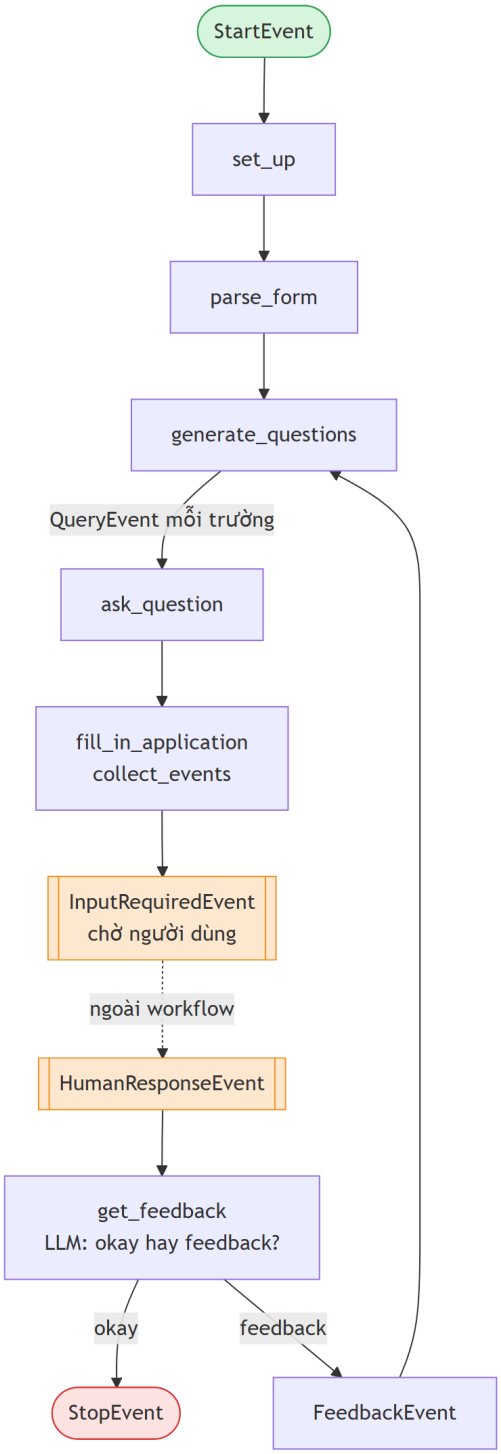
## 6.1 Hai sự kiện đặc biệt

Ta dùng hai sự kiện đặc biệt được thiết kế riêng để *thoát* khỏi workflow lấy phản hồi rồi *quay lại*:

- `InputRequiredEvent` – phát ra từ bên trong workflow, báo rằng cần nhập liệu từ bên ngoài.
- `HumanResponseEvent` – gửi *vào* workflow từ bên ngoài, mang theo phản hồi của người dùng.

## 6.2 Tách bước

Trước đây một bước duy nhất vừa parse đơn vừa bắn hết câu hỏi. Vì giờ có thể lặp lại nhiều lần, ta không cần parse đơn mỗi lần, nên tách thành các bước riêng – kiểu tái cấu trúc rất thường gặp khi workflow phức tạp dần. Thêm một `FeedbackEvent` cho vòng lặp góp ý (Hình 6.1).




Hình 6.1: Workflow human-in-the-loop: tạm dừng ở `InputRequiredEvent`, chờ `HumanResponseEvent` từ bên ngoài, rồi quyết định dừng hay lặp lại.

```
from llama_index.core.workflow import InputRequiredEvent, HumanResponseEvent

class FeedbackEvent(Event):
    feedback: str

class GenerateQuestionsEvent(Event):
    pass
```

```
class RAGWorkflow(Workflow):
    @step
    async def parse_form(
        self, ctx: Context, ev: ParseFormEvent
    ) -> GenerateQuestionsEvent:
        # ... parse đơn -> fields
        await ctx.set("fields_to_fill", fields)
        return GenerateQuestionsEvent()

    @step
    async def generate_questions(
        self, ctx: Context, ev: GenerateQuestionsEvent | FeedbackEvent
    ) -> QueryEvent:
        fields = await ctx.get("fields_to_fill")
        for field in fields:
            question = f"How would you answer this question about the " \
                       f"candidate? <field>{field}</field>"
            ctx.send_event(QueryEvent(field=field, query=question))
        await ctx.set("total_fields", len(fields))
        return
```

`generate_questions` có thể được kích hoạt *hoặc* bởi `GenerateQuestionsEvent` (từ bước parse), *hoặc* bởi `FeedbackEvent` (vòng lặp sau khi nhận góp ý).

## 6.3 Tạm dừng chờ phản hồi

Bước `fill_in_application` nay phát `InputRequiredEvent` để hỏi người dùng, rồi một bước mới `get_feedback` chờ `HumanResponseEvent`.

```
    @step
    async def fill_in_application(
        self, ctx: Context, ev: ResponseEvent
    ) -> InputRequiredEvent:
        total_fields = await ctx.get("total_fields")
        responses = ctx.collect_events(ev, [ResponseEvent] * total_fields)
        if responses is None:
            return None

        responseList = "\n".join(
            "Field: " + r.field + "\n" + "Response: " + r.response
            for r in responses)
        result = self.llm.complete(f"""
            You are given a list of fields in an application form and responses
            to questions about those fields from a resume. Combine the two into
            a list of fields and succinct, factual answers to fill in those
            fields.

            <responses>
            {responseList}
            </responses>
        """)
        # lưu kết quả để dùng lại sau
        await ctx.set("filled_form", str(result))

        return InputRequiredEvent(
            prefix="How does this look? Give me any feedback you have on "
```

```
                    "any of the answers.",
                result=result,
            )

        @step
        async def get_feedback(
            self, ctx: Context, ev: HumanResponseEvent
        ) -> FeedbackEvent | StopEvent:
            result = self.llm.complete(f"""
                You have received some human feedback on the form-filling task
                you've done. Does everything look good, or is there more work to be
                done?
                <feedback>
                {ev.response}
                </feedback>
                If everything is fine, respond with just the word 'OKAY'.
                If there's any other feedback, respond with just the word 'FEEDBACK'.
            """)

            verdict = result.text.strip()

            print(f"LLM says the verdict was {verdict}")
            if verdict == "OKAY":
                return StopEvent(result=await ctx.get("filled_form"))
            else:
                return FeedbackEvent(feedback=ev.response)
```

Lưu ý điểm đặc biệt: `InputRequiredEvent` *không* được bước nào trong workflow bắt lấy, còn `HumanResponseEvent` *không* được phát ra từ bên trong workflow. Cả hai nối với thế giới bên ngoài.

## 6.4 Lồng phản hồi từ bên ngoài

Cách lấy phản hồi giống hệt cách ta xử lý streaming trước đây: `InputRequiredEvent` là một sự kiện trong luồng, ta chặn nó, rồi dùng `ctx.send_event` gửi lại một `HumanResponseEvent`.

```
w = RAGWorkflow(timeout=600, verbose=False)
handler = w.run(
    resume_file="data/fake_resume.pdf",
    application_form="data/fake_application_form.pdf",
)

async for event in handler.stream_events():
    if isinstance(event, InputRequiredEvent):
        print("We've filled in your form! Here are the results:\n")
        print(event.result)
        # giờ hỏi phản hồi từ bàn phím
        response = input(event.prefix)
        handler.ctx.send_event(
            HumanResponseEvent(response=response)
        )

response = await handler
print("Agent complete! Here's your final result:")
print(str(response))
```

Khi chạy, workflow in bản điền hiện tại; trường **projects portfolio** vẫn là một danh sách thay vì một đường link. Ta gõ góp ý `"Portfolio should be a URL"`. LLM đọc góp ý và kết luận verdict là

`FEEDBACK`, kích hoạt vòng lặp. Nếu gõ góp ý tích cực như `"That's great"`, LLM kết luận `OKAY` và workflow dừng.

**So khớp verdict chính xác**

Lưu ý bước `get_feedback` lấy `result.text.strip()` rồi so sánh *bằng* (`verdict == "OKAY"`), chứ không phải kiểm tra chuỗi con. Vì vậy prompt yêu cầu LLM trả về đúng *một từ* `OKAY` hoặc `FEEDBACK` là quan trọng – nếu LLM trả về cả câu, nhánh so khớp sẽ sai.

## 6.5 Thực sự dùng phản hồi

Ở bản vừa rồi, vòng lặp mới chỉ *chạy lại* mà chưa thực sự *dùng* góp ý. Giờ ta sửa `generate_questions` để kiểm tra có góp ý không rồi gắn nó vào mỗi câu hỏi. Ví dụ đơn giản này gắn góp ý vào *mọi* câu hỏi phòng khi có liên quan; một agent tinh vi hơn chỉ gắn vào đúng trường mà góp ý nhắm tới.

```
    @step
    async def generate_questions(
        self, ctx: Context, ev: GenerateQuestionsEvent | FeedbackEvent
    ) -> QueryEvent:
        fields = await ctx.get("fields_to_fill")
        for field in fields:
            question = f"How would you answer this question about the " \
                       f"candidate? <field>{field}</field>"

            # Mới! Có feedback không? Nếu có thì gắn vào query:
            if hasattr(ev, "feedback"):      # đây là FeedbackEvent
                question += f"""
                    \nWe previously got feedback about how we answered the
                    questions. It might not be relevant to this particular
                    field, but here it is:
                    <feedback>{ev.feedback}</feedback>
                """

            ctx.send_event(QueryEvent(field=field, query=question))
        await ctx.set("total_fields", len(fields))
        return
```

Ta kiểm tra `hasattr(ev, "feedback")`: nếu có thì đây là `FeedbackEvent` và ta chèn góp ý vào, kèm lời nhắc rằng góp ý có thể không liên quan tới trường này để LLM khỏi bối rối. Chạy lại và góp ý `"Portfolio should be a URL"`, lần này trường portfolio được đổi đúng thành một URL. Một điểm thú vị khi góp ý bằng ngôn ngữ tự nhiên: bạn nói gì cũng được, LLM tự hiểu ý. Agent giờ đã phản hồi được góp ý của con người và cho ra bản điền chính xác hơn.

# Chương 7

# Dùng giọng nói

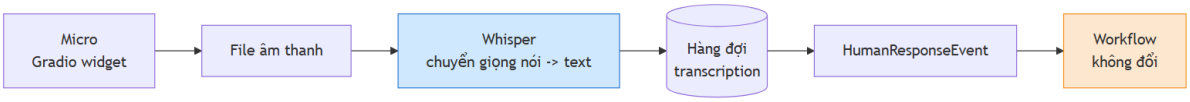
Chương trước ta góp ý cho agent bằng văn bản. Chương này góp ý bằng **lời nói**: thêm năng lực **đa phương thức** để agent bắt được phản hồi dạng âm thanh do người dùng nói ra. Điều hay là *workflow không cần đổi gì* – ta chỉ thay đường vào của phản hồi từ bàn phím sang micro.

## 7.1 Nhìn lại workflow

Đưa workflow human-in-the-loop ở chương trước qua trình trực quan hoá, ta thấy toàn bộ *happy path*: bắt đầu, thiết lập, parse đơn, sinh câu hỏi, hỏi, điền đơn, lấy phản hồi của con người – rồi hoặc dừng, hoặc đóng vòng lặp để làm lại. Đây đúng là workflow của chương trước, không có gì mới.

## 7.2 Phiên âm bằng Whisper

Ta dùng mô hình **Whisper** của OpenAI. LlamaIndex có sẵn cách phiên âm file âm thanh thành text qua Whisper (Hình 7.1).




Hình 7.1: Micro (Gradio) ghi âm, Whisper phiên âm, kết quả vào hàng đợi rồi gửi vào workflow dưới dạng `HumanResponseEvent`.

```
from llama_index.readers.whisper import WhisperReader
import gradio as gr

def transcribe_speech(filepath):
    if filepath is None:
        gr.Warning("No audio found, please retry.")
    audio_file = open(filepath, "rb")
    reader = WhisperReader(
        model="whisper-1",
        api_key=openai_api_key,
    )
    documents = reader.load_data(filepath)
    return documents[0].text
```

Hàm nhận một file, nạp nó, khởi tạo `WhisperReader` với mô hình `whisper-1` rồi lấy text ra. Đây cũng chính là loại `Document` ta đã dùng cho RAG ở các chương trước.

## 7.3 Bắt âm thanh với Gradio

Để có âm thanh, ta dùng **Gradio** – có widget render ngay trong notebook để thu âm từ micro. Khi thu xong, nó gọi `transcribe_speech` trên dữ liệu đã ghi.

```
transcription_value = ""   # biến toàn cục giữ transcription

def store_transcription(output):
    global transcription_value
    transcription_value = output
    return output

mic_transcribe = gr.Interface(
    fn=lambda x: store_transcription(transcribe_speech(x)),
    inputs=gr.Audio(sources="microphone", type="filepath"),
    outputs=gr.Textbox(label="Transcription"),
)

test_interface = gr.Blocks()
with test_interface:
    gr.TabbedInterface([mic_transcribe], ["Transcribe Microphone"])

test_interface.launch(
    share=False,
    server_port=8000,
    prevent_thread_lock=True,
)
# ... sau khi lấy xong transcription:
# print(transcription_value)
# test_interface.close()
```

Gradio dựng sẵn giao diện micro, lắng nghe và lưu transcription vào biến toàn cục chỉ trong vài dòng code. Luồng thao tác trên widget là *record* → *stop* → *submit*. Nhớ gọi `test_interface.close()` để tắt giao diện Gradio cũ trước khi mở cái mới, nếu không các giao diện (cùng dùng cổng 8000) sẽ xung đột với nhau.

## 7.4 Lớp xử lý transcription

Để nối gọn vào workflow, ta gói logic trên vào một lớp `TranscriptionHandler`. Khác biệt chính: lưu kết quả vào một **hàng đợi** (queue) thay vì biến toàn cục, và *poll* hàng đợi mỗi 0.5 giây để biết khi nào có kết quả.

```
import asyncio
from queue import Queue

class TranscriptionHandler:
    # tạo một hàng đợi giữ các giá trị transcription
    def __init__(self):
        self.transcription_queue = Queue()
        self.interface = None

    # mỗi lần ghi được gì thì bỏ vào hàng đợi
    def store_transcription(self, output):
        self.transcription_queue.put(output)
        return output
```

```
    # cùng giao diện + logic như trên, nhưng lưu vào hàng đợi thay vì biến toàn cục
    def create_interface(self):
        mic_transcribe = gr.Interface(
            fn=lambda x: self.store_transcription(transcribe_speech(x)),
            inputs=gr.Audio(sources="microphone", type="filepath"),
            outputs=gr.Textbox(label="Transcription"),
        )
        self.interface = gr.Blocks()
        with self.interface:
            gr.TabbedInterface([mic_transcribe], ["Transcribe Microphone"])
        return self.interface

    async def get_transcription(self):
        self.interface = self.create_interface()
        self.interface.launch(
            share=False,
            server_port=8000,
            prevent_thread_lock=True,
        )
        # poll mỗi 1.5 giây cho tới khi có gì đó trong hàng đợi
        while True:
            if not self.transcription_queue.empty():
                result = self.transcription_queue.get()
                if self.interface is not None:
                    self.interface.close()
                return result
            await asyncio.sleep(1.5)
```

Vòng `while True` ngủ 1.5 giây mỗi lượt; khi có gì trong hàng đợi, nó đóng giao diện và trả về kết quả.

## 7.5 Dùng giọng nói thay bàn phím

Giờ chỉ cần thay `input()` bằng `TranscriptionHandler` ở chỗ lấy phản hồi của con người. **Workflow không đổi gì cả.**

```
w = RAGWorkflow(timeout=600, verbose=False)
handler = w.run(
    resume_file="./data/fake_resume.pdf",
    application_form="./data/fake_application_form.pdf",
)

async for event in handler.stream_events():
    if isinstance(event, InputRequiredEvent):
        # lấy transcription thay cho input() bàn phím
        transcription_handler = TranscriptionHandler()
        response = await transcription_handler.get_transcription()

        handler.ctx.send_event(
            HumanResponseEvent(response=response)
        )

response = await handler
print("Agent complete! Here's your final result:")
print(str(response))
```

Khi chạy, agent in từng câu hỏi và câu trả lời khi đang suy nghĩ (bước `ask_question` có thêm `print`), rồi dựng một giao diện Gradio xin phản hồi. Nhìn các câu trả lời, ta thấy nó lại mắc lỗi cũ: trường portfolio là một danh sách thay vì URL. Ta nói vào micro: *"The portfolio field should be a URL"*. LLM đọc transcript, kết luận đây là góp ý, và chạy lại với góp ý gắn vào mỗi câu hỏi.

Lần này portfolio được điền đúng thành URL. Ta nói tiếp *"That's great, good job"*; LLM hiểu đây là phản hồi tích cực, kết luận `OKAY` và xuất ra bản điền cuối cùng.

**Tổng kết**

Ta đã xây một tác tử xử lý tài liệu hoàn chỉnh: dùng RAG đọc CV, parse mẫu đơn thành câu hỏi, điền đơn, nhận góp ý của con người bằng cả văn bản lẫn giọng nói, và lặp lại cho tới khi đạt kết quả mong muốn. Điểm cốt lõi là kiến trúc **event-driven** của workflow: thêm năng lực mới (như giọng nói) chỉ là thay đường vào/ra của sự kiện, không phải viết lại logic bên trong.

## 7.6 Tài liệu tham khảo thêm

Để tìm hiểu sâu hơn về agentic document workflows, xem bài viết giới thiệu của LlamaIndex (`https://www.llamaindex.ai/blog/introducing-agentic-document-workflows`) và các ví dụ cài đặt tại kho `run-llama/llamacloud-demo` (`https://github.com/run-llama/llamacloud-demo`).